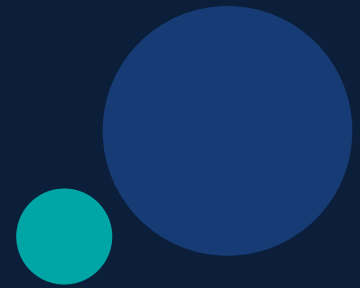


SECURITY GATES

Enterprise DevSecOps Project



A professional deep-dive document defining CI/CD security quality gates, failure thresholds, evidence requirements, remediation workflow, exception handling, and governance model for the Enterprise DevSecOps lab.

Author Jagriti Banerjee	Document security-gates	Project Enterprise DevSecOps Security Lab	Environment Private VM / Kind / GitHub Actions
----------------------------	----------------------------	---	--



Security gates convert individual tool outputs into pass/fail decisions before promotion to the next delivery stage.

Document classification: portfolio-ready technical documentation for a controlled private lab. No third-party systems were targeted.

1. Executive Summary

This document defines the security gate strategy used in the Enterprise DevSecOps project. A security gate is a controlled checkpoint in the delivery lifecycle where security evidence is evaluated before the application, container image, Kubernetes manifest, or runtime configuration is allowed to move forward. The purpose is to make security measurable, repeatable, and auditable rather than dependent on manual review alone.

The project uses a deliberately vulnerable Flask application and a hardened Kubernetes deployment path to demonstrate how security gates can identify risk early, block unsafe promotion, provide evidence for reviewers, and drive remediation. The gate model combines application security testing, dependency analysis, container scanning, Kubernetes configuration validation, supply chain verification, GitOps drift control, admission control, and runtime monitoring.

Gate decisions in this document are grouped into three modes: blocking gates, warning gates, and evidence-only gates. Blocking gates prevent promotion when unacceptable risk is identified. Warning gates allow continuation but require tracking and remediation. Evidence-only gates collect proof for audit, portfolio, and security reporting.

Gate Domain	Primary Tools	Decision Type	Security Value
Application Security	Bandit, manual AppSec tests	Block / Warn	Detects unsafe patterns such as command execution, unsafe rendering, and SQL injection-prone code.
Dependency Security	pip-audit	Block / Warn	Identifies vulnerable Python packages before build or deployment.
Container Security	Docker, Trivy	Block / Warn	Validates image vulnerability posture and Docker hardening assumptions.
Kubernetes Security	Trivy config, kubect!, Gatekeeper	Block	Prevents unsafe manifests and enforces baseline deployment controls.
Supply Chain	Syft, Cosign, GHCR	Block / Evidence	Provides SBOM, image signature, and artifact integrity evidence.
GitOps	ArgoCD	Block / Evidence	Detects drift and restores cluster state to the Git-defined baseline.
Runtime	Falco, Prometheus, Grafana	Detect / Alert	Provides detection and observability after deployment.

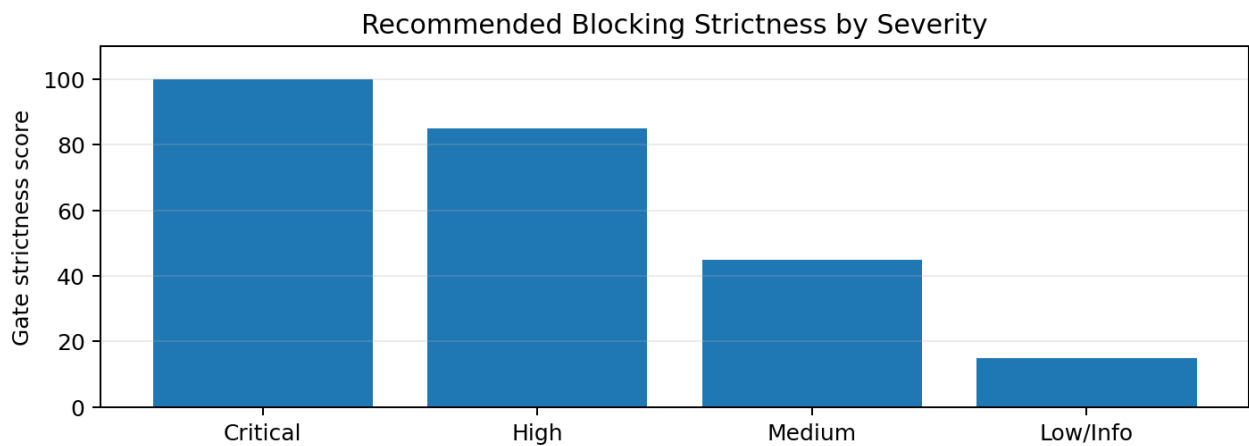
2. Security Gate Operating Model

The security gate model follows a simple rule: each control must produce evidence, each evidence item must map to a decision, and each failed decision must produce a remediation action. This prevents scanner output from becoming passive noise and turns security tooling into a delivery-quality control system.

2.1 Gate Decision Types

Decision Type	Meaning	Recommended Use
Pass	No policy-breaking issue is detected or all issues are below the configured threshold.	Allow promotion to the next pipeline stage.
Fail	A blocking risk is detected and the pipeline should not proceed.	Critical vulnerabilities, confirmed injection risk, exposed secrets, unsigned images, or invalid Kubernetes security controls.
Warn	Issue is valid but not severe enough to block the current stage.	Medium findings, non-production misconfigurations, or exceptions pending remediation.
Manual Review	Automated evidence is inconclusive and requires human decision.	False positive review, temporary acceptance, compensating controls, or business exception.
Evidence Only	The control is used for reporting and assurance but not enforced as a blocker.	Runtime metrics, dashboards, SBOM record keeping, and portfolio documentation.

2.2 Recommended Severity Thresholds



Critical findings should be treated as hard failures. High findings should normally block deployment unless an approved exception exists. Medium findings should generate remediation tickets with target dates. Low and informational findings should be tracked for hygiene and maturity improvement.

3. Security Gate Inventory

The following inventory defines the core gates implemented or recommended for the project. These gates reflect the path from source code to CI/CD execution, container packaging, Kubernetes deployment, GitOps control, and runtime monitoring.

ID	Gate	Stage	Control Objective	Default Outcome
SG-01	Repository Hygiene and Branch Protection	Source	Protect main branch, require review, prevent uncontrolled changes.	Block merge if required checks fail.
SG-02	Secrets Detection	Source / CI	Prevent secrets, tokens, private keys, and credentials from entering Git.	Block if verified secret is detected.
SG-03	SAST - Python Code Security	CI	Detect unsafe code patterns in Flask application.	Block on high-confidence high-severity findings.
SG-04	SCA - Python Dependency Audit	CI	Detect known vulnerabilities in requirements.txt dependencies.	Block on Critical/High fixable vulnerabilities.
SG-05	Secure Docker Build	Build	Ensure image is built reproducibly and runs as non-root.	Block if build fails or runtime identity check fails.
SG-06	Container Image Vulnerability Scan	Build / Registry	Scan image packages and OS layers.	Block on Critical/High depending on environment.
SG-07	Filesystem and Secret Scan	CI	Scan repository and image context for leaks or risky artifacts.	Block on secrets, warn on low-risk findings.
SG-08	Kubernetes Manifest Security	Pre-deploy	Validate securityContext, RBAC, NetworkPolicy, probes, and resource limits.	Block on unsafe workload configuration.
SG-09	SBOM Generation	Build / Registry	Generate software inventory for packages and image components.	Evidence required before release.
SG-10	Image Signing and Verification	Registry / Deploy	Ensure deployed image can be cryptographically verified.	Block if signature is missing or invalid.
SG-11	Admission Policy Enforcement	Kubernetes	Block non-compliant Kubernetes objects at admission time.	Block non-compliant deployment.
SG-12	GitOps Sync and Drift Control	Deploy	Ensure live cluster matches Git-defined desired state.	Self-heal or flag OutOfSync.
SG-13	Runtime Detection and Observability	Runtime	Detect suspicious container and Kubernetes activity.	Alert and investigate.

4. Gate Architecture and Control Flow

The project uses layered security gates rather than a single scanner. This provides defense-in-depth and ensures that failure in one layer can be compensated by another. For example, a vulnerable dependency can be detected during CI, unsafe Kubernetes configuration can be blocked before deployment, and suspicious runtime activity can still be detected after deployment.

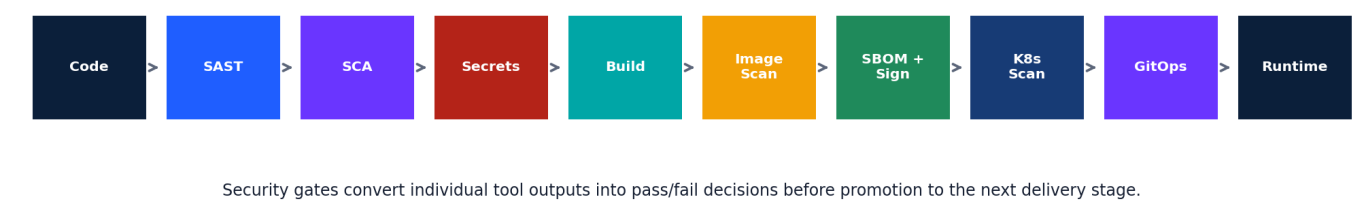


Figure 1. Security gate flow across the DevSecOps lifecycle.

Flow Step	Gate Responsibility	Evidence Produced
Developer commit	Repository hygiene, branch protection, secret scanning	Git commit history, workflow run, secret scan output
CI security tests	SAST, SCA, filesystem scan	Bandit JSON/report, pip-audit report, Trivy FS report
Container build	Build image, validate runtime user, scan image	Docker build logs, non-root proof, Trivy image report
Supply chain	Generate SBOM, sign image, verify attestation	Syft SBOM, Cosign signature, verification logs
Deployment	Manifest scan, policy-as-code, GitOps sync	Trivy config report, Gatekeeper admission output, ArgoCD sync status
Runtime	Threat detection and monitoring	Falco alerts, Prometheus queries, Grafana dashboard screenshots

5. Detailed Gate Definitions

SG-01: Repository Hygiene and Branch Protection

Objective: Ensure the default branch cannot be modified without review and passing security checks.

Tools / controls: GitHub branch protection, required status checks, signed commits where possible.

Gate Input	Decision Logic	Output Evidence
Pull request to main, direct push attempt, workflow status change.	Fail if required security jobs do not pass or if review requirements are not met.	GitHub repository settings, branch rule screenshot, workflow check history.

Remediation expectation: Require PR review, enforce status checks, disallow force pushes, and document exceptions.

SG-02: Secrets Detection Gate

Objective: Prevent accidental exposure of API tokens, private keys, passwords, cloud keys, and registry credentials.

Tools / controls: Gitleaks or equivalent secret scanning integrated into CI and local pre-commit checks.

Gate Input	Decision Logic	Output Evidence
New commits, pull requests, and workflow runs.	Fail on verified secrets or high-confidence credential patterns. Warn on test fixtures after review.	Secret scan report, commit hash, remediation note, and credential rotation record.

Remediation expectation: Remove secret from Git, rotate exposed credential, purge history if needed, and add safe sample files.

SG-03: SAST Gate - Bandit for Python

Objective: Detect insecure Python coding patterns in the vulnerable Flask application before merge or deployment.

Tools / controls: Bandit SAST scan running locally and in GitHub Actions.

Gate Input	Decision Logic	Output Evidence
Changes under src/app, CI workflow run, pull request update.	Fail on high-confidence high-severity issues. Warn on medium issues until triaged.	Bandit terminal output, JSON artifact, GitHub Actions job evidence.

Remediation expectation: Replace unsafe code with secure alternatives, add regression tests, and re-run Bandit.

SG-04: SCA Gate - pip-audit for Dependencies

Objective: Detect known vulnerabilities in Python packages used by the Flask application.

Tools / controls: pip-audit scan against requirements.txt.

Gate Input	Decision Logic	Output Evidence
Dependency changes, scheduled scan, pull request, CI workflow run.	Fail on Critical/High vulnerabilities with available fixed versions. Warn on unfixed or non-exploitable issues after review.	pip-audit report, package version diff, remediation evidence.

Remediation expectation: Upgrade package versions, pin secure versions, test application compatibility, and capture retest output.

SG-05: Secure Docker Build Gate

Objective: Ensure the container image can be built reproducibly and follows runtime hardening expectations.

Tools / controls: Docker build, Dockerfile review, non-root identity check.

Gate Input	Decision Logic	Output Evidence
Dockerfile change or application build.	Fail if image build fails, container runs as root, or runtime healthcheck fails.	Docker build output, image list, non-root user proof.

Remediation expectation: Use multi-stage build, fixed non-root UID/GID, no unnecessary packages, and healthcheck validation.

SG-06: Container Image Vulnerability Gate

Objective: Identify vulnerable OS packages and application dependencies inside the container image.

Tools / controls: Trivy image scan.

Gate Input	Decision Logic	Output Evidence
Image built or pushed to GHCR.	Fail on Critical/High findings based on environment policy and exploitability. Warn on accepted risks.	Trivy image scan report, CI artifact, fixed image digest.

Remediation expectation: Patch base image, upgrade dependencies, rebuild image, and rescan.

SG-07: Repository Filesystem and Secret Scan Gate

Objective: Scan the repository and build context for secrets, vulnerable files, and risky configuration.

Tools / controls: Trivy filesystem scan, optional Gitleaks.

Gate Input	Decision Logic	Output Evidence
Pull request, push to main, release candidate.	Fail on confirmed secret leakage. Warn on low-risk file hygiene issues.	Trivy FS report, secret scan output, remediation note.

Remediation expectation: Remove sensitive file, add .gitignore rules, regenerate secrets, and update evidence.

SG-08: Kubernetes Manifest Security Gate

Objective: Detect insecure Kubernetes deployment configuration before application of manifests.

Tools / controls: Trivy config scan, manual Kubernetes hardening checklist.

Gate Input	Decision Logic	Output Evidence
Change in k8s/base manifests or GitOps path.	Fail if manifests lack non-root execution, resource limits, read-only filesystem, or privilege controls.	Trivy config report, kubectl describe output, manifest diff.

Remediation expectation: Patch YAML with secure securityContext, RBAC, NetworkPolicy, probes, and resource controls.

SG-09: SBOM Generation Gate

Objective: Create a software inventory that supports vulnerability management and supply chain review.

Tools / controls: Syft SPDX/CycloneDX SBOM generation.

Gate Input	Decision Logic	Output Evidence
Image release candidate or registry push.	Fail release if SBOM is missing for release image. Evidence-only for local experiments.	SBOM JSON, SBOM table report, package count summary.

Remediation expectation: Regenerate SBOM from final image digest and store it in the repository or registry attachment.

SG-10: Image Signing and Verification Gate

Objective: Cryptographically prove that the image is the expected artifact produced by the trusted workflow.

Tools / controls: Cosign sign and verify.

Gate Input	Decision Logic	Output Evidence
Image push, pre-deploy validation, release candidate.	Fail if signature verification fails or public key cannot validate the image.	Cosign verify output, signature record, image digest.

Remediation expectation: Rebuild, sign using approved key, verify signature, and rotate compromised keys if needed.

SG-11: Admission Policy Gate - OPA Gatekeeper

Objective: Prevent non-compliant Kubernetes resources from entering the cluster.

Tools / controls: OPA Gatekeeper ConstraintTemplates and Constraints.

Gate Input	Decision Logic	Output Evidence
kubectl apply, ArgoCD sync, or any Kubernetes admission request.	Fail if required labels, pod security expectations, or policy conditions are missing.	Gatekeeper denial output, allowed deployment proof, policy YAML.

Remediation expectation: Update manifest to satisfy policy or create approved exception with owner and expiry.

SG-12: GitOps Drift Gate - ArgoCD

Objective: Ensure live Kubernetes state matches the desired state defined in Git.

Tools / controls: ArgoCD automated sync, self-heal, prune.

Gate Input	Decision Logic	Output Evidence
Manual cluster change, Git commit, drift event, sync operation.	OutOfSync state requires correction. Self-heal should restore intended state when enabled.	ArgoCD synced/healthy screenshot, drift self-heal evidence.

Remediation expectation: Commit desired changes to Git; do not make permanent manual cluster changes.

SG-13: Runtime Detection Gate - Falco and Observability

Objective: Detect suspicious activity after deployment and provide runtime assurance.

Tools / controls: Falco, Falcosidekick, Prometheus, Grafana.

Gate Input	Decision Logic	Output Evidence
Container runtime events, sensitive file access, privileged behavior, suspicious shell activity.	Alert and investigate. Runtime detections do not block CI, but they require triage and incident-style evidence.	Falco logs, Prometheus query, Grafana dashboard evidence.

Remediation expectation: Add preventive control, update Falco rules, review RBAC/pod security, and document closure.

6. Gate Threshold Matrix

Security gates must be predictable. The following threshold matrix defines how findings should affect pipeline or deployment decisions. A clear threshold prevents inconsistent handling of findings and makes the report easier for recruiters, security engineers, and reviewers to understand.

Finding Type	Example	Default Gate Action	Exception Rules
Confirmed Critical vulnerability	Remote code execution, critical image CVE, privileged host access pattern	Block	Only temporary exception with documented owner, expiry, and compensating control.
Confirmed High vulnerability	Command injection, SQL injection, high dependency CVE	Block for main/release	Exception requires risk acceptance and remediation deadline.
Medium vulnerability	Missing header, non-critical dependency issue, hardening gap	Warn / ticket	May proceed with backlog item if non-production.
Low / Informational	Version disclosure, minor hygiene issue	Record	No blocking unless repeated or policy-specific.
Verified secret	Token, private key, password	Block immediately	No exception; rotate and remove.
Unsigned image	No valid Cosign signature	Block deployment	Exception only for local lab proof-of-concept.
SBOM missing	No SBOM for release image	Block release	Allow local test but not production-style release.
Admission policy violation	Missing required labels, privileged pod, unsafe context	Block	Fix YAML or create formal policy exception.
Runtime detection	Falco sensitive file access alert	Alert / investigate	Requires triage result and closure evidence.

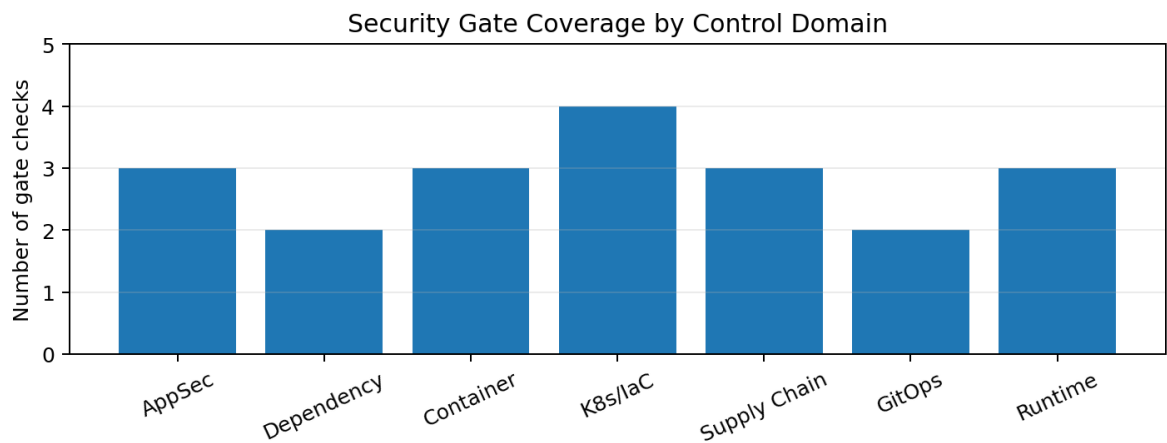


Figure 2. Gate coverage by domain across the Enterprise DevSecOps lifecycle.

7. CI/CD Security Gate Implementation

In the project, GitHub Actions acts as the automated enforcement layer for code, dependency, filesystem, image, and registry gates. The pipeline separates jobs so that failures are easy to diagnose and evidence artifacts can be downloaded per gate.

7.1 Recommended Pipeline Job Order

Job	Purpose	Gate Output
checkout	Pull repository code into the runner.	Commit SHA and repository context.
sast	Run Bandit on Python application code.	Bandit report and SAST terminal evidence.
dependency-scan	Run pip-audit against requirements.txt.	Dependency vulnerability report.
filesystem-scan	Run Trivy against repository filesystem.	Filesystem vulnerability/misconfig/secret scan report.
docker-build-and-scan	Build the container image and run Trivy image scan.	Image scan report and optional SARIF.
push-to-ghcr	Publish image only after previous gates pass.	Registry image tag and digest.
supply-chain	Generate SBOM and sign/verify image.	SBOM, Cosign verification, attestation evidence.

7.2 Example Gate Logic

```
# Example gate logic for CI/CD policy
# Critical/High findings should block promotion unless explicitly accepted.

bandit -r src/app -f json -o security/reports/bandit-report.json
pip-audit -r src/app/requirements.txt -f json -o security/reports/pip-audit-report.json
trivy fs . --scanners vuln,secret,misconfig --severity CRITICAL,HIGH --exit-code 1
trivy image ghcr.io/<owner>/<repo>:latest --severity CRITICAL,HIGH --exit-code 1
cosign verify --key security/cosign/cosign.pub ghcr.io/<owner>/<repo>:latest
trivy config k8s/base --severity CRITICAL,HIGH --exit-code 1
```

The sample above uses fail-fast behavior for high-risk categories. In the lab, some jobs may be set to exit-code 0 for demonstration so that reports can be generated without stopping learning workflows. For a production-style pipeline, release gates should use non-zero exit codes on unacceptable risk.

8. Evidence Requirements

Every gate must produce evidence that can be reviewed independently. Evidence should be stored in the repository under security/reports, evidence/screenshots, or GitHub Actions artifacts. The purpose is to support auditability, retest proof, and recruiter-facing technical credibility.

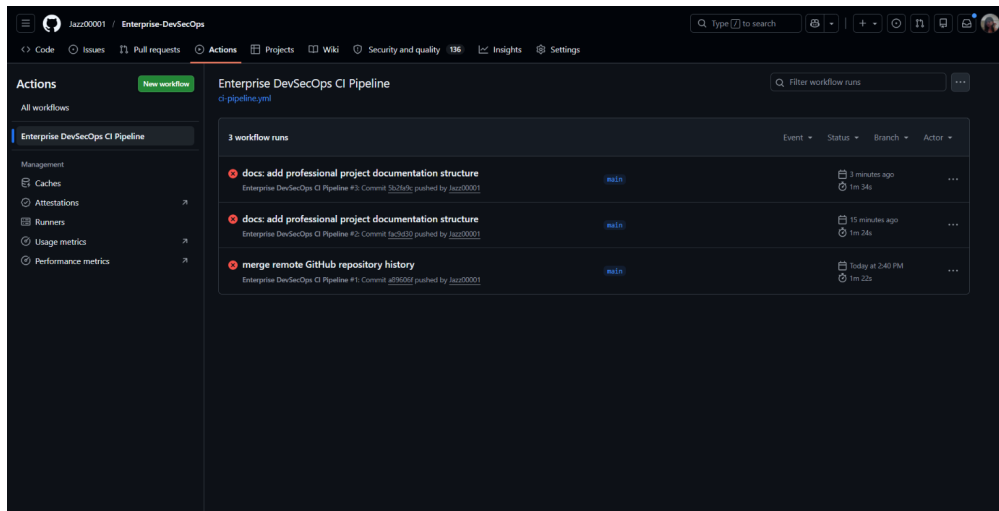
Gate	Evidence Artifact	Recommended Location	Reviewer Question Answered
SAST	Bandit JSON and CI job screenshot	security/reports/bandit-report.json	Which insecure code patterns were identified?
SCA	pip-audit JSON and job output	security/reports/pip-audit-report.json	Which dependencies require upgrades?
Image Scan	Trivy image report	security/reports/trivy-image-report.txt	Does the final container image contain known vulnerabilities?
Manifest Scan	Trivy config report	security/reports/trivy-k8s-manifest-scan.txt	Are Kubernetes manifests hardened?
Supply Chain	Cosign verify output, SBOM, attestation logs	security/reports, security/sbom	Can the image and SBOM be verified?
Admission	Gatekeeper allow/deny output	evidence/screenshots	Are policy violations blocked before runtime?
GitOps	ArgoCD synced/healthy and drift screenshots	evidence/screenshots	Does live state match Git state?
Runtime	Falco logs, Prometheus query, Grafana dashboard	security/reports, evidence/screenshots	Can suspicious behavior be detected after deployment?

Evidence should avoid exposing secrets. Screenshots should crop or redact credentials, tokens, private keys, or sensitive environment variables. For command output, collect enough context to prove the result without leaking confidential data.

9. Selected Gate Evidence

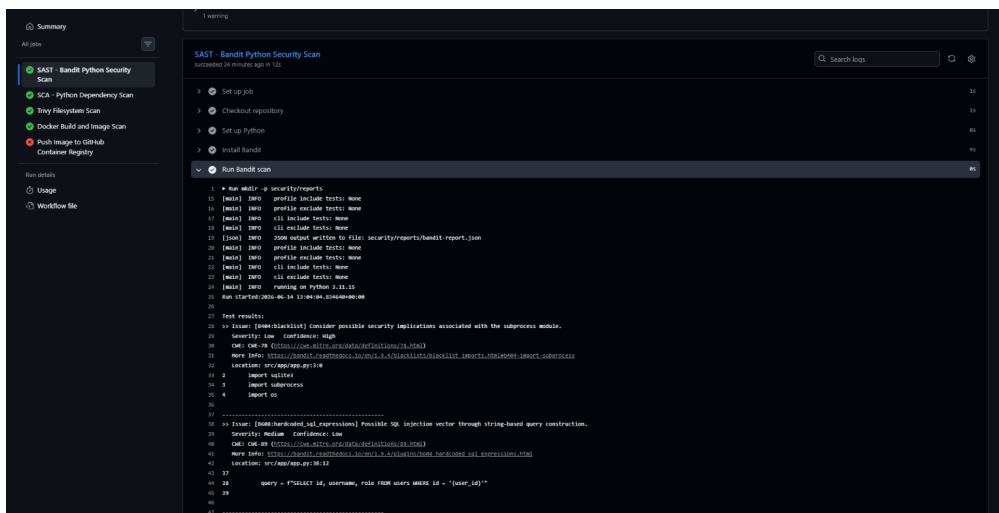
The following selected screenshots demonstrate gate execution and evidence capture from the private Enterprise DevSecOps lab.

GitHub Actions pipeline running



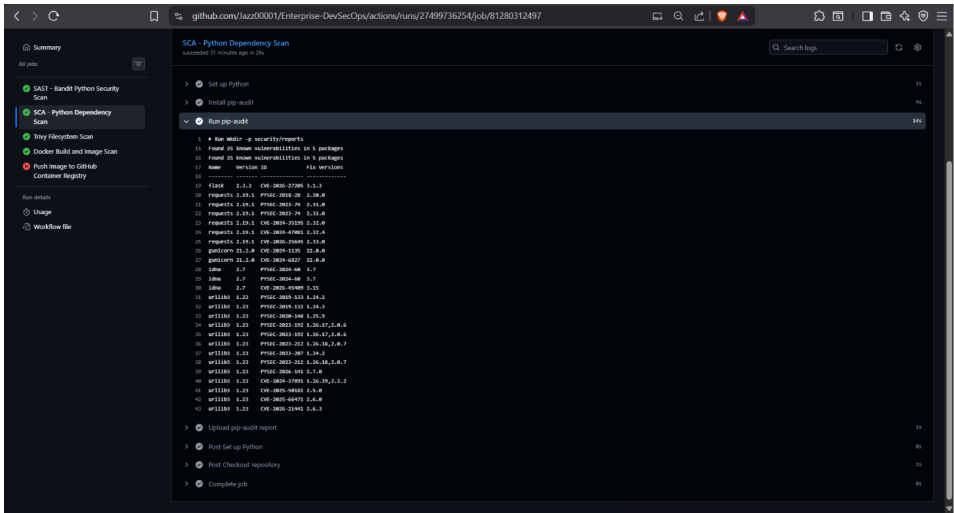
Evidence: 057-github-actions-pipeline-running.png

Bandit SAST job output



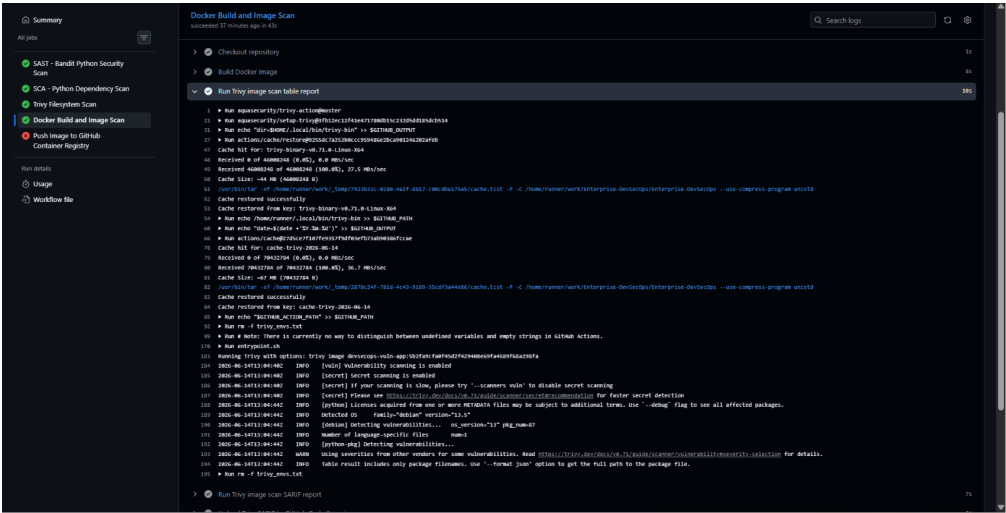
Evidence: 059-bandit-job-output.png

pip-audit dependency job output



Evidence: 061-pip-audit-job-output.png

Trivy image scan job



Evidence: 064-trivy-image-scan-job.png

Kubernetes manifest scan



```
jazz@ubuntu-endpoint: ~ - ssh - Windows PowerShell
20 [
21 {
  runAsUser: 10001
  resources: {}

KSV-0110 (LOW): deployment secure-devsecops-app in default namespace should
set metadata.namespace to a non-default namespace

Checks whether a workload is running in the default namespace.

See https://avd.aquasec.com/misconfig/ksv-0110

secure-demo-deployment.yaml:3-4
3 {
4   metadata:
5     name: secure-devsecops-app

KSV-0118 (HIGH): deployment secure-devsecops-app in default namespace is usi
ng the default security context, which allows root privileges

Security context controls the allocation of security parameters for the pod/
container/volume, ensuring the appropriate level of protection. Relying on d
efault security context may expose vulnerabilities to potential attacks that
rely on privileged access.

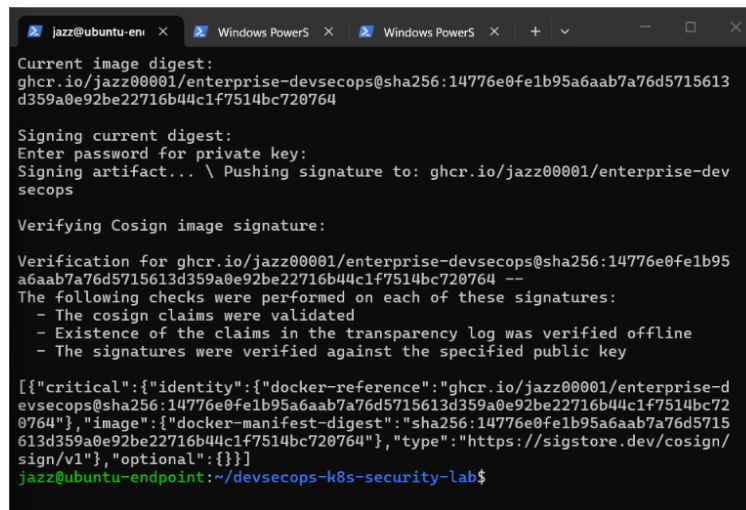
See https://avd.aquasec.com/misconfig/ksv-0118

secure-demo-deployment.yaml:14-21
14 spec:
15   containers:
16   - name: app
17     image: devsecops-vuln-app:lab
18     securityContext:
19       privileged: false
20       runAsUser: 10001
21     resources: {}

jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
```

Evidence: 088-trivy-k8s-manifest-scan.png

Cosign signature verification



```
jazz@ubuntu-eni X Windows PowerShell X Windows PowerShell X + -
Current image digest:
ghcr.io/jazz00001/enterprise-devsecops@sha256:14776e0fe1b95a6aab7a76d5715613
d359a0e92be22716b44c1f7514bc720764

Signing current digest:
Enter password for private key:
Signing artifact... \ Pushing signature to: ghcr.io/jazz00001/enterprise-dev
secops

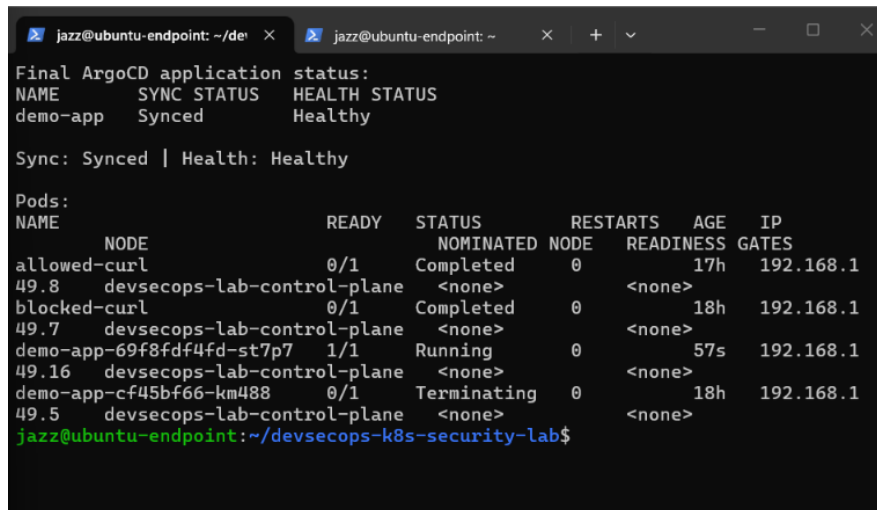
Verifying Cosign image signature:

Verification for ghcr.io/jazz00001/enterprise-devsecops@sha256:14776e0fe1b95
a6aab7a76d5715613d359a0e92be22716b44c1f7514bc720764 --
The following checks were performed on each of these signatures:
- The cosign claims were validated
- Existence of the claims in the transparency log was verified offline
- The signatures were verified against the specified public key

[{"critical":{"identity":{"docker-reference":"ghcr.io/jazz00001/enterprise-d
evsecops@sha256:14776e0fe1b95a6aab7a76d5715613d359a0e92be22716b44c1f7514bc72
0764"},"image":{"docker-manifest-digest":"sha256:14776e0fe1b95a6aab7a76d5715
613d359a0e92be22716b44c1f7514bc720764"},"type":"https://sigstore.dev/cosign/
sign/v1"},"optional":{}}}]
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
```

Evidence: 115-cosign-signature-verified.png

ArgoCD synced and healthy



```

jazz@ubuntu-endpoint: ~/dev
jazz@ubuntu-endpoint: ~
Final ArgoCD application status:
NAME      SYNC STATUS  HEALTH STATUS
demo-app  Synced      Healthy

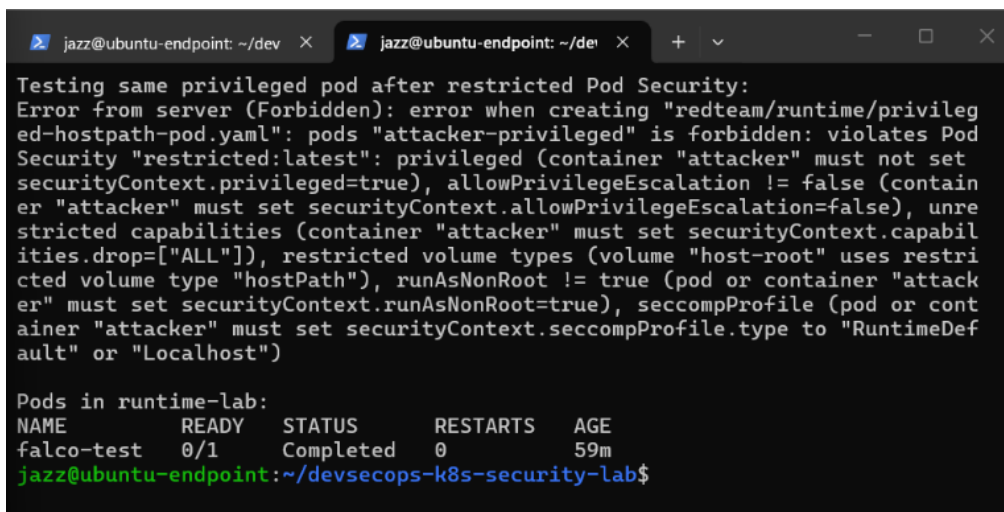
Sync: Synced | Health: Healthy

Pods:
NAME      NODE      READY  STATUS      RESTARTS  AGE  IP
49.8      devsecops-lab-control-plane  0/1      Completed   0      17h  192.168.1
blocked-curl  0/1      Completed   0      18h  192.168.1
demo-app-69f8fdf4fd-st7p7  1/1      Running     0      57s  192.168.1
49.16     devsecops-lab-control-plane  <none>    <none>
demo-app-cf45bf66-km488  0/1      Terminating 0      18h  192.168.1
49.5      devsecops-lab-control-plane  <none>    <none>
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$

```

Evidence: 097-argocd-app-synced-healthy.png

Pod Security blocks privileged pod



```

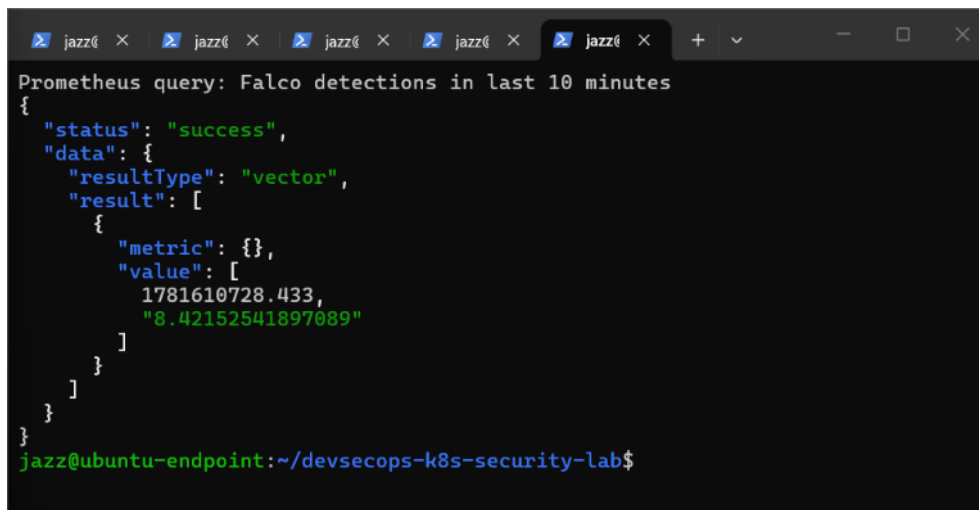
jazz@ubuntu-endpoint: ~/dev
jazz@ubuntu-endpoint: ~/dev
Testing same privileged pod after restricted Pod Security:
Error from server (Forbidden): error when creating "redteam/runtime/privileged-hostpath-pod.yaml": pods "attacker-privileged" is forbidden: violates Pod Security "restricted:latest": privileged (container "attacker" must not set securityContext.privileged=true), allowPrivilegeEscalation != false (container "attacker" must set securityContext.allowPrivilegeEscalation=false), unrestricted capabilities (container "attacker" must set securityContext.capabilities.drop=["ALL"]), restricted volume types (volume "host-root" uses restricted volume type "hostPath"), runAsNonRoot != true (pod or container "attacker" must set securityContext.runAsNonRoot=true), seccompProfile (pod or container "attacker" must set securityContext.seccompProfile.type to "RuntimeDefault" or "Localhost")

Pods in runtime-lab:
NAME      READY  STATUS      RESTARTS  AGE
falco-test 0/1     Completed   0          59m
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$

```

Evidence: 138-privileged-pod-blocked-by-pod-security.png

Prometheus Falco query

A terminal window with a dark background and light-colored text. The window title bar shows five tabs, each labeled 'jazz@'. The terminal content shows a Prometheus query for Falco detections in the last 10 minutes. The output is a JSON object with a 'status' of 'success' and a 'data' object containing a 'result' array. The 'result' array has one element with a 'metric' object and a 'value' array containing two numbers: 1781610728.433 and 8.42152541897089. The prompt at the bottom is 'jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab\$'.

```
jazz@ ubuntu-endpoint:~/devsecops-k8s-security-lab$  
Prometheus query: Falco detections in last 10 minutes  
{  
  "status": "success",  
  "data": {  
    "resultType": "vector",  
    "result": [  
      {  
        "metric": {},  
        "value": [  
          1781610728.433,  
          "8.42152541897089"  
        ]  
      }  
    ]  
  }  
}  
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Evidence: 156-prometheus-falco-rule-matches-query.png

10. Gate Failure Handling and Remediation Workflow

A failed gate should not be treated as an isolated scanner problem. It should trigger a formal remediation workflow that includes triage, ownership, fix implementation, retest, and closure evidence.

Step	Activity	Output
1	Identify gate failure and capture raw evidence.	Workflow log, report file, screenshot.
2	Triage severity, exploitability, confidence, and affected asset.	Validated finding or false positive decision.
3	Assign owner and remediation target.	Ticket, backlog item, or documented owner.
4	Implement secure code, dependency, container, or manifest fix.	Pull request or local lab commit.
5	Run the same gate again using the original test case.	Retest output.
6	Record status as remediated, accepted risk, false positive, or deferred.	Closure note and evidence index update.

10.1 Exception Handling

An exception should be rare, explicit, time-bound, and supported by compensating controls. For this project, exceptions are acceptable only for lab demonstration mode where the purpose is to generate evidence of an intentionally vulnerable state. In production-style workflows, exceptions should include a business owner, technical owner, expiry date, affected component, reason for acceptance, and a planned remediation date.

10.2 False Positive Handling

False positives should be documented with the rule ID, tool, evidence, analysis, and reason for dismissal. Suppression should be narrow and code-specific. Broad suppression of entire scanners or directories should be avoided because it weakens the security gate model.

11. Gate-Based Remediation Backlog

Priority	Backlog Item	Mapped Gate	Expected Security Improvement
P1	Replace vulnerable Flask SQL query with parameterized query.	SG-03	Removes SQL injection attack path.
P1	Replace shell=True command execution with safe subprocess list invocation and input allowlist.	SG-03	Removes command injection attack path.
P1	Remove unsafe render_template_string pattern and use static templates with escaped variables.	SG-03	Reduces template injection and unsafe rendering risk.
P1	Upgrade vulnerable Python dependencies and pin secure versions.	SG-04	Reduces known dependency exposure.
P2	Enable mandatory Trivy severity threshold for CI release branch.	SG-06	Prevents vulnerable images from release promotion.
P2	Require SBOM generation for each release image digest.	SG-09	Improves software inventory and audit readiness.
P2	Require valid Cosign signature before deployment.	SG-10	Improves image integrity and provenance assurance.
P2	Expand Gatekeeper rules beyond labels to enforce pod security context.	SG-11	Blocks unsafe deployments at admission.
P3	Add dashboards for gate failure trends and remediation time.	SG-13	Improves management visibility.

11.1 Suggested SLA Model

Severity	Remediation Target	Escalation
Critical	Immediate fix or block release	Security owner and project owner review
High	Fix before merge/release	Security gate owner approval required for exception
Medium	Backlog remediation with target sprint	Track in issue register
Low	Best-effort hygiene improvement	Review during hardening cycle

12. Metrics and Maturity Indicators

Security gates should be measured like engineering quality controls. The following metrics help demonstrate maturity and make the project easier to explain in interviews.

Metric	Why It Matters	How to Capture
Gate pass rate	Shows how often changes meet security expectations.	Count passed vs failed CI jobs.
Mean time to remediate	Measures responsiveness to security findings.	Track time from finding creation to retest pass.
Critical/High open findings	Indicates release risk.	Aggregate Bandit, pip-audit, Trivy, and admission failures.
Exception count	Tracks risk acceptance discipline.	Maintain exception register with expiry.
SBOM coverage	Shows supply chain inventory completeness.	Count release images with SBOM attached.
Signed image coverage	Measures artifact integrity coverage.	Verify Cosign signatures for release images.
GitOps drift events	Measures configuration stability.	ArgoCD app status and audit history.
Runtime alert volume	Shows runtime threat detection and tuning status.	Falco metrics in Prometheus and Grafana.

Maturity should improve over time: early lab gates may run in reporting mode, but advanced gates should become blocking once false positives are reviewed and the remediation workflow is stable.

13. Roles and Responsibilities

Role	Responsibilities
Application owner	Fix application vulnerabilities, review SAST results, approve code remediation.
DevSecOps engineer	Maintain CI/CD gate logic, scanner configuration, artifact upload, and workflow reliability.
Platform/Kubernetes owner	Maintain Kubernetes hardening, Gatekeeper policies, NetworkPolicies, GitOps sync, and runtime controls.
Security reviewer	Validate findings, approve exceptions, review remediation evidence, and update risk status.
Release approver	Confirm that required gates pass before promoting a release candidate.

In this private lab, Jagriti Banerjee acts as the author, implementer, validator, and security reviewer. For enterprise adoption, these duties should be separated to avoid self-approval of high-risk changes.

14. Recommended Future Enhancements

Enhancement	Security Benefit
Add branch protection rules and required status checks in GitHub	Prevents merge without passing security gates.
Add Semgrep with custom rules for Flask patterns	Improves AppSec detection coverage beyond Bandit.
Add Dependabot or scheduled SCA scans	Keeps dependency vulnerability detection continuous.
Add Kyverno or Sigstore Policy Controller image verification	Enforces signature verification at Kubernetes admission time.
Add SARIF uploads for scanner results	Improves GitHub code scanning visibility.
Add ticket automation for failed gates	Improves remediation accountability.
Add environment-specific thresholds	Allows stricter production-style rules and more flexible lab/dev rules.

15. Conclusion

The security-gates model transforms the Enterprise DevSecOps project from a collection of tools into a governed, evidence-driven delivery workflow. Each gate has a purpose, decision rule, evidence requirement, and remediation path. This structure demonstrates practical understanding of how modern security programs integrate with CI/CD pipelines, Kubernetes deployment, supply chain assurance, and runtime monitoring.

For portfolio and interview purposes, this document shows that the project is not only technically implemented, but also documented in a way that reflects enterprise security thinking: risk thresholds, gate ownership, evidence management, exception handling, retest proof, and continuous improvement.

Final status: The Enterprise DevSecOps project includes security gates across application code, dependencies, container image, Kubernetes manifests, supply chain controls, GitOps deployment, admission policy, and runtime observability.